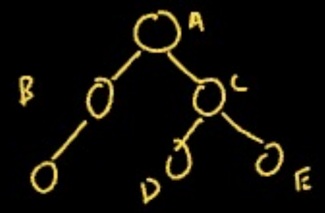


Binary tree: A tree where each node has a max of 2 children.

Perfect Binary tree: All the levels of the binary tree are full.

ht = #levels − 1

length of the longest path from the root to any of its leaves.

For a perfect binary tree

$$\#\text{nodes} = \underbrace{2^0 + 2^1 + 2^2 + \cdots + 2^h}_{\#\text{terms} = h+1}$$

$$= \frac{2^{h+1} - 1}{2-1} = 2^{h+1} - 1$$

GP sum $= \dfrac{a(r^n - 1)}{(r-1)}$ (n → #terms)

$$\boxed{n = 2^{h+1} - 1}$$

$$\Rightarrow 2^{h+1} = n+1 \longrightarrow \log_2(n+1) = h+1$$

$$\Rightarrow \log_2 2^{h+1} = \log_2(n+1)$$

$$\Rightarrow (h+1)\log_2 2 = \log_2(n+1)$$

$$\Rightarrow (h+1) = \log_2(n+1)$$

$$\Rightarrow \boxed{h = \log_2(n+1) - 1}$$

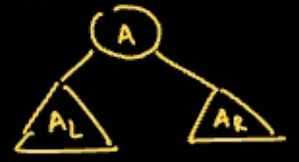
$$\boxed{h = \Theta(\log n)}$$

$\log_b b^n = n$

$\log_b b = 1$

$\log_b 1 = 0$

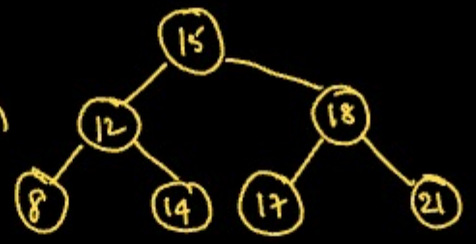
Binary Search trees



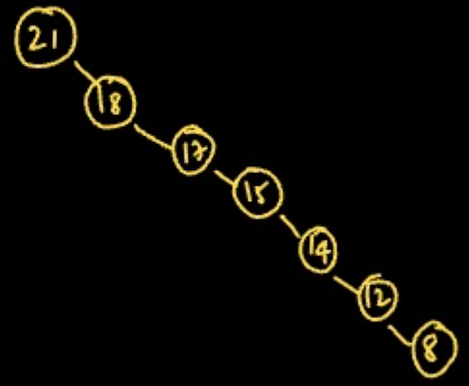
all vals in A_L < A's val

all vals in A_R > A's val

$h = \Theta(\lg n)$



$h = \Theta(n)$



Heaps

- Complete Binary trees
  1. Balanced

     $h = \Theta(\log n)$
  2. stored using arrays

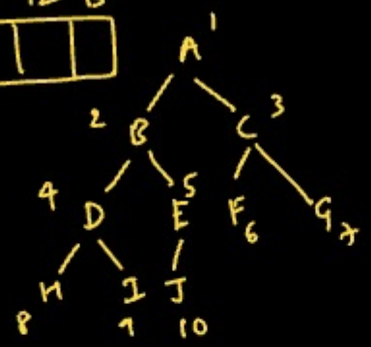
Complete Binary tree:

- All levels are full except possibly the last level
- Last level must be filled from left to right

Complete? ✓ Complete? ✗ Complete? ✓

int[] → 4n bytes

	1	2	3	4	5	6	7	8	9	10	11	12	13
-	A	B	C	D	E	F	G	H	I	J			



index of parent(i) = $\lfloor i/2 \rfloor$

left child(i) = $2*i$

right child(i) = $2*i+1$

TreeNode

```
int val;
TreeNode left;
TreeNode right;
TreeNode par;
```

4 bytes
8 bytes
8 bytes

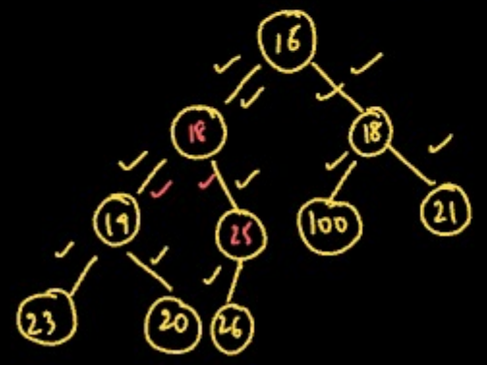
padding: 4 bytes

total size: 24 bytes

24n bytes for n nodes

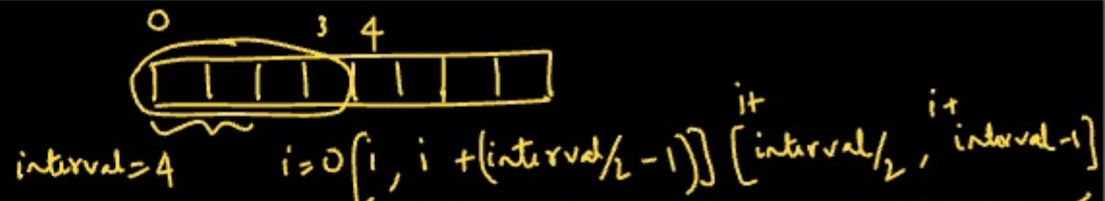
- swim()
- sink()

- <u>Min</u> heaps:
  - Property: For every node, its value should be $\geq$ the parent's value

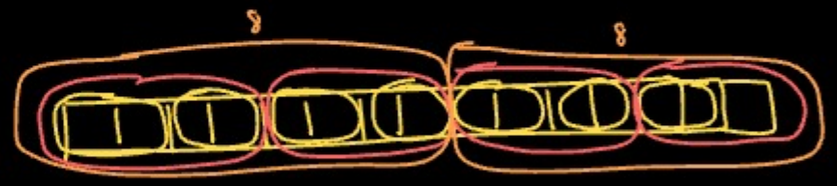


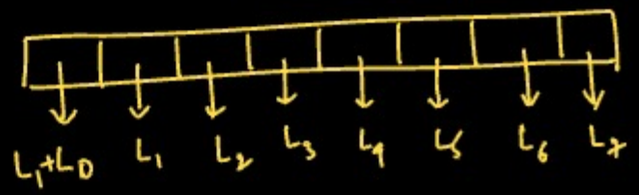
Invariants
1. Heap property
2. Complete Binary Tree ✓

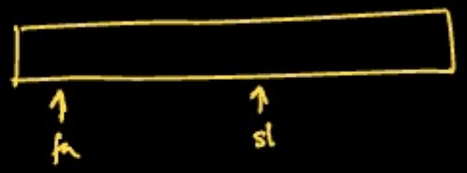
15



i=





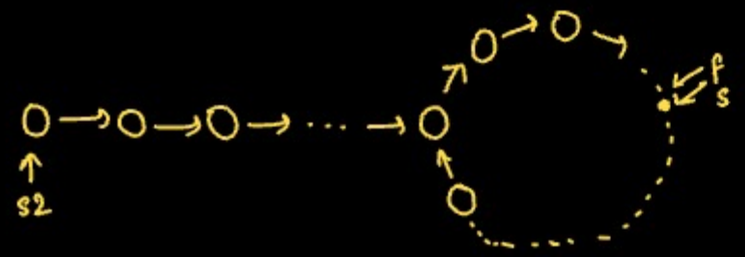


0 1 2 3 4

nums [3, 1, 3, 4, 2]

n+1 integers

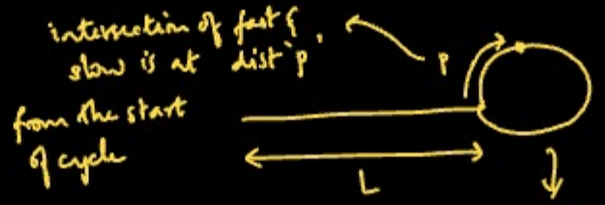
[1, n]



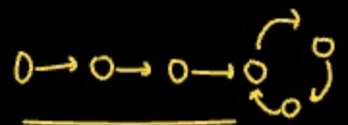
Algo:

1. Run fast & slow ptrs until they meet
2. Start another slow ptr at the head & move slow & slow2 one step at a time until they meet.

The meeting point of slow & slow2 is the start of the cycle



length (# hops) of cycle is 'k'



L=3 (# hops in the path)

$1 \le k \le n$

$0 \le p < k$

We do s iterations of slow ptr

$s = L + tk + p$

After 's' iterations, how far from the start would fast be?

$= 2s$

$= 2[L + tk + p]$ $\qquad \underline{L} + \underline{ck} + \underline{p}$

$= 2L + 2tk + 2p$

$= L + (L + p + 2tk) + p$

if this part is a multiple of k, then they meet

$L + p + 2tk$ = multiple of k

$\Rightarrow$ $\boxed{L+p \text{ has to be a multiple of } k}$

$L + p = ck$

$\Rightarrow L = ck - p$

$= \underbrace{(c-1)k + k}_{ck} - p$

$$\boxed{L = (c-1)k + (k-p)}$$

State Machines

- States: $\{q\}$
- Initial state: q_0
- Transitions: $\{q \to r\}$
- Reachability:
  - A state is reachable from the initial state if there exists a path from the initial state through valid transitions
- Invariant:
  - A property that holds true for every reachable state.
  - state is reachable $\Rightarrow$ Invariant is true for that state

    If invariant is false $\Rightarrow$ not reachable

- Sorted 2-sum:
  - State: $\{(l, r)\}$
  - Initial state: $(0, n-1)$
  - Transitions:

$$(l, r) \to (l+1, r) \quad \text{if} \quad a[l] + a[r] < \text{target}$$

$$(l, r) \to (l, r-1) \quad \text{if} \quad a[l] + a[r] > \text{target}$$

$$p \mapsto q$$

$$\downarrow$$

- $a[l] + a[r] < \text{target}$
- solution is contained in $\underline{\underline{[l, r]}}$

$$\begin{cases} a[l] + a[r] < \text{target} \\ a[l] + a[i] < \text{target} \\ \quad \text{for all } i \text{ in } [l+1, r-1] \end{cases}$$

l is not part of the solution

$\therefore$ we can increment 'l'

Number Representations

- $758 = 700 + 50 + 8$

00758 $\quad = \underline{7}\cdot 10^2 + \underline{5}\cdot 10^1 + \underline{8}\cdot 10^0$

Base 10 ⇒ digits: 0, 1, 2, ..., 9
Base 2 ⇒ " : 0, 1
Base 3 ⇒ " : 0, 1, 2
Base 16 ⇒ : 0, 1, 2, ..., 10, 11, 12, 13, 14, 15

A B C D E F

↓ Hexadecimal

0x9DEF

123
123

$0x12 = 1\times 16^1 + 2\times 16^0$
$= 18_{10}$

- Binary to decimal

543210
110010 $= 1\cdot 2^5 + 1\cdot 2^4 + 0\cdot 2^3 + 0\cdot 2^2 + 1\cdot 2^1 + 0\cdot 2^0$
$= 32 + 16 + 2 = \underline{50}$

- Decimal to binary

50
↓ -2^5
18
↓ -2^4
2
↓ -2^1
0

1	1	0	0	1	0
5	4	3	2	1	0

- Decimal to base 3

315
↓ -243
72
↓ $-2\cdot 27$
18
↓ $-2\cdot 9$
0

1	0	2	2	0	0
5	4	3	2	1	0

$3^2(\ k\ ) + 3^1 + 3^0$

$3^0 = 1$
$3^1 = 3$
$3^2 = 9$
$3^3 = 27$
$3^4 = 81$
$3^5 = 243$

- Binary & Hexadecimal

0 - 0000		8	- 1000
1 - 0001		9	- 1001
2 - 0010		10(A)	- 1010
3 - 0011		11(B)	- 1011
4 - 0100		12(C)	- 1100
5 - 0101		13(D)	- 1101
6 - 0110		14(E)	- 1110
7 - 0111		15(F)	- 1111

110 0111

0x 6 7

$0\cdot 2^7 + 1\cdot 2^6 + 1\cdot 2^5 + 0\cdot 2^4$
$+ 0\cdot 2^3 + 1\cdot 2^2 + 1\cdot 2^1 + 1\cdot 2^0$

$16^1\ 2^4\ (0\cdot 2^3 + 1\cdot 2^2 + 1\cdot 2^1 + 0\cdot 2^0)$
$16^0\ +2^0\ (0\cdot 2^3 + 1\cdot 2^2 + 1\cdot 2^1 + 1\cdot 2^0)$

- Integer representations
  - int - 32 bits
    short - 16 bits
    long - 64 bits
  - unsigned & signed
    - unsigned ↓ we have only non-negative values
    - signed ↓ we represent +ve & -ve values & 0

3 2 1 0
1 1 0 1

$-1.2^3 + 1.2^2 + 0.2^1 + 1.2^0$

1111 = 15
+ 1
10000 = 0
×

4 bits:

	Unsigned int	Sign & magnitude	One's complement	Two's complement
0000	0	0	0	0
0001	1	1	1	1
0010	2	2	2	2
0011	3	3	3	3
0100	4	4 ✓	4	4
0101	5	5	5	5
0110	6	6	6	6
0111	7	7	7	7
1000	8	-0	-7	-8
1001	9	-1	-6	-7
1010	10	-2	-5	-6
1011	11	-3	-4	-5
1100	12	-4	-3	-4
1101	13	-5	-2	-3
1110	14	-6	-1	-2
1111	15	-7 ✓	-0	-1

k bits → range of unsigned values

$[0, 2^k - 1]$

unsigned int (32 bits) : $[0, 2^{32} - 1]$

unsigned long (64 bits) : $[0, 2^{64} - 1]$

	7 → 0111	+1
-8 =	8 → 1000	
-7 =	9 → 1001	+1
-6 =	10 → 1010	+1

-7	1001
+3	0011
-4	1100
7	0111
3	0011
	1010

4 bits → Unsigned range: $[0, 15]$ → $2^4 - 1$

→ signed range: $[-8, +7]$

32 bits ⇒ representation of smallest # = $\overset{31}{1}\,\overset{30}{0}\,\overset{29}{0} \cdots \overset{1}{0}\,\overset{0}{0} = -2^{31}$

representation of largest # = $0\,1\,1 \cdots 1\,1 = 2^{31} - 1$

$[-2^{31}, 2^{31} - 1]$

$2^{30} + 2^{29} + \cdots + 2^1 + 2^0$

$\frac{1\,[2^{31} - 1]}{2 - 1} = 2^{31} - 1$

16 bit: $[-2^{15}, 2^{15} - 1]$

$[0, 2^{16} - 1]$

int - 4 bits → $[-8, 7]$

lo = 5 hi = 7 → mid = 6

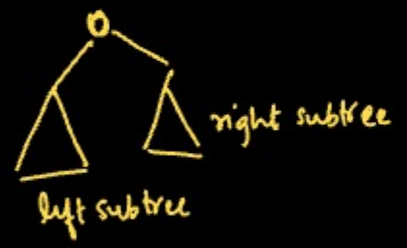
mid = (lo + hi)/2

= (5+7)/2 = 12

$$\boxed{mid = lo + \frac{(hi - lo)}{2}}$$

Trees

- Most of the questions can be solved using recursion

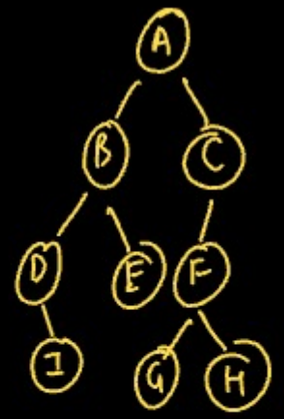


wrapper fn

```
def binarySearch(nums, key):
    return binarySearchR(nums, key, 0, len(nums)-1)

def binarySearchR(nums, key, lo, hi):
    mid =
        bsR(nums, key, lo, mid-1)
        bsR( "    "  , mid+1, hi)
        return mid
```

recursive function



Preorder

A B D I E C F G H

- A (pre(B)) pre(C)
- A(B pre(D) pre(E)) pre(C)

⋮

Postorder

I D E B G H F C A

Inorder

D I B E A G F H C

Pre order
↓
process node first then subtrees

Post order
↓
process subtrees first then node

in order

process left subtree
process node
process right subtree

[A] [B C] [D E F] [I G H]

level order traversal
(BFS in graphs)

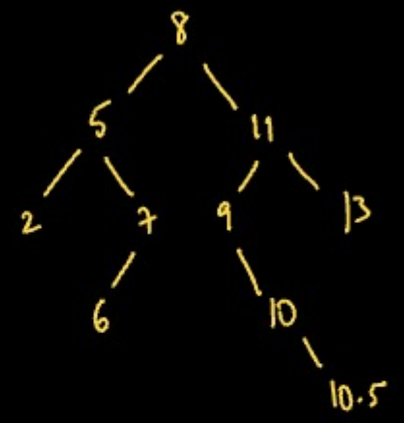
FIFO → Queue

A B C D E F I G H

as long as queue is not empty:
remove element from queue
(process it)
add its children from l to r into the queue

BST- Binary Search Trees

- assuming no duplicates
- BST property
  - for every node, all elements in its left subtree < node's value
    & all elements in its right subtree > node's value

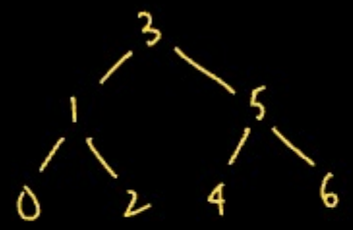


Inorder: 2 5 6 7 8 9 10 10.5 11 13

↓

of BSTs gives us the sorted order of values

- find(), add(), delete(), ...
  → $O(h)$ → height of the tree → for balanced BSTs, it is $O(\log n)$

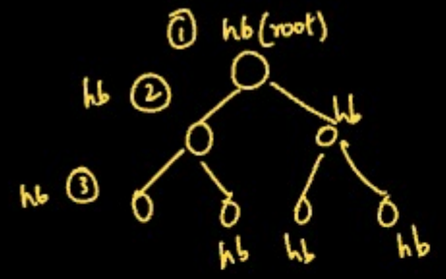


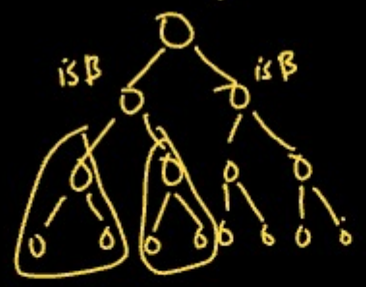
BST? ✓

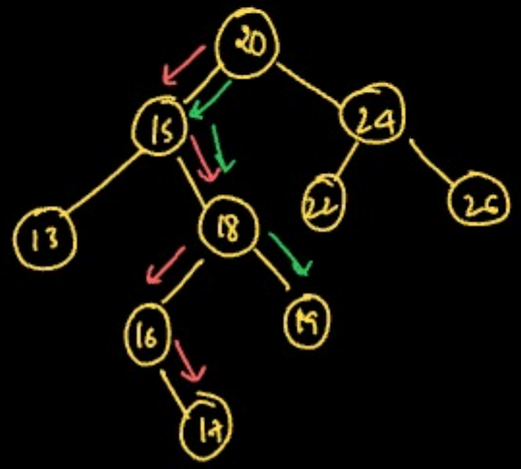
BST? ✓

Balanced binary trees
iff $h = \Theta(\log n)$

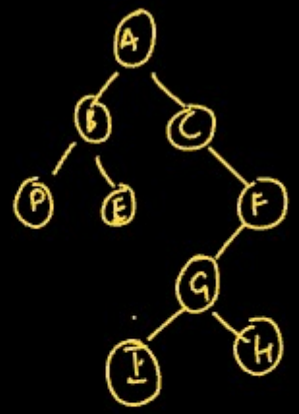
AVL trees (height balanced - diff between hts of subtrees ≤ 1 for all nodes)
Red Black Trees etc







LCA(17, 19)



right_node = []

queue

* →

while (!queue.isEmpty()):

// queue will exactly have nodes from one level

level_nodes = list(queue)

for loop for len(q) times

process (node)

deque

↓

double ended queue in O(1)

inclusive

list[a:b]

not inclusive

↓

length of slice = b − a

$i=10$ $\quad i=10$

$\downarrow$

1 2 3 4 (

$res = 0$

$res = res*10 + 1 = 1$

$res = res*10 + 2 = 12$

$res = res*10 + 3 = 123$

$res = res*10 + 4 = 1234$

Horner's method

ax^3+bx^2+cx+d

$1\cdot 10^3 + 2\cdot 10^2 + 3\cdot 10^1 + 4\cdot 10^0$

$(((a)x+b)x+c)x+d$

$\ldots\ (112(\ )(\text{right tree}))\ldots$

$\boxed{112}$

$\Theta(n^d) \Rightarrow a_n x^n + a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \cdots + a_2x^2 + a_1x + a_0$

$3x^2 - 5 \qquad 4x^{12}$